

Contents

1	Bayes, cadenas de Markov y modelos ocultos	1
1.1	De medir a decidir	1
1.2	Probabilidad condicional: la pieza que faltaba	1
1.3	Clasificar con Bayes	2
1.3.1	El Teorema de Bayes	2
1.3.2	El supuesto «ingenuo» (Naïve Bayes)	3
1.3.3	Regla de decisión y ejemplos resueltos	3
1.3.4	El problema de la frecuencia cero y el subdesbordamiento numérico	4
1.4	Cadenas de Markov: recuperar el orden	5
1.4.1	La propiedad de Markov de orden 1	5
1.4.2	Definición formal y validación de la matriz de transición	5
1.4.3	Trazas de cálculo y matrices inválidas	6
1.4.4	Explosión de parámetros y detección de islas CpG	7
1.5	Del estado observado al oculto (HMM)	8
1.5.1	Los tres problemas clásicos de los HMM	8
1.5.2	Definición del modelo HMM para promotores	8
1.6	Trazar Viterbi	9
1.6.1	El Algoritmo de Viterbi (Programación Dinámica)	9
1.6.2	Traza completa sobre la secuencia ‘ATGC’	10
1.6.3	El peligro del clasificador posicional ingenuo	11
1.7	La operación peligrosa: matrices no normalizadas y ceros	11
1.8	Actividad de depuración: errores en Viterbi	12
1.9	Síntesis operativa y tablas de diagnóstico	12
1.9.1	Tabla de diagnóstico de síntomas en modelos probabilísticos	12
1.9.2	Tabla de síntesis con preguntas de control	12
1.10	Glosario conceptual	13
1.11	Conexión con el curso y siguientes pasos	13
1.12	Fuentes y reproducibilidad	13
	Anexo práctico · Clasificar, modelar y decodificar con probabilidad	15

Chapter 1

Bayes, cadenas de Markov y modelos ocultos

1.1 De medir a decidir

Medir el tiempo o el coste de un algoritmo (TC-CH03–TC-CH06) no responde cuánto orden o incertidumbre contiene una secuencia; contar frecuencias y calcular la entropía de Shannon (TC-CH07) sí cuantifica la diversidad estática de los símbolos, pero no dice nada sobre qué mecanismo biológico los generó ni sobre la dependencia de orden entre posiciones sucesivas.

Esencial

Este capítulo da el salto metodológico fundamental: pasar de **medir** a **decidir** y **modelar**:

- **Clasificar:** Dada una secuencia desconocida de ADN, ¿pertenece a una región codificante de un gen o a un intrón no codificante? (**Clasificador Naïve-Bayes**).
- **Modelar transiciones:** ¿Cómo influye el nucleótido anterior en la probabilidad del siguiente para distinguir islas CpG de la heterocromatina? (**Cadenas de Markov**).
- **Decodificar estados biológicos ocultos:** Si la función genómica (promotor vs fondo intergénico) no viene etiquetada directamente en la molécula, ¿cómo reconstruir la secuencia más probable de estados que emitió los nucleótidos observados? (**Modelos Ocultos de Markov – HMM y Algoritmo de Viterbi**).

Contar frecuencias y calcular entropía describe una distribución estática de símbolos, pero no dice qué mecanismo genera esos símbolos ni si existe dependencia entre posiciones sucesivas. Bayes, Markov y los modelos ocultos añaden, sucesivamente: una regla de decisión entre clases; una hipótesis de generación con memoria de un símbolo; y un mecanismo generador cuyo estado no se observa directamente.

Esencial

La ruta **esencial** cubre los fundamentos de la probabilidad condicional y regla de la cadena, el clasificador Naïve-Bayes (incluyendo el suavizado de Laplace y el cálculo en log-espacio), las cadenas de Markov de orden 1 (con validación estricta de matrices de transición), los modelos ocultos de Markov (HMM) y el algoritmo de decodificación de Viterbi con su traza sobre ‘ATGC’; 110–130 minutos. La ruta de **estudio** profundiza en el análisis de la explosión de parámetros en órdenes superiores, la comparación entre la decodificación global de Viterbi frente al clasificador posicional ingenuo, la actividad de depuración y el anexo práctico; 60–80 minutos adicionales.

1.2 Probabilidad condicional: la pieza que faltaba

Para modelar dependencias se necesita la **probabilidad condicional**, $P(A | B)$ —la probabilidad de que ocurra el suceso A dado que ya se conoce con certeza la ocurrencia del suceso

B —:

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad \text{con } P(B) > 0$$

La probabilidad condicional $P(A | B)$ es la probabilidad de que ocurra el suceso A sabiendo que ya ha ocurrido el suceso B ; es en general distinta de la probabilidad no condicionada $P(A)$, porque incorpora la información de que B ya se sabe cierto.

Dos sucesos son **independientes** si y solo si conocer uno no altera en absoluto la probabilidad del otro: $P(A | B) = P(A)$. Si $P(A | B) \neq P(A)$, el suceso B aporta información predictiva sobre A , y asumir independencia destruye la estructura informativa real de la secuencia.

Dos sucesos A y B son independientes cuando $P(A | B) = P(A)$: saber que ocurrió B no cambia la probabilidad de A . Cuando $P(A | B) \neq P(A)$, B aporta información real sobre A y tratarlos como independientes pierde esa información.

Regla del producto y Regla de la cadena. De la definición condicional se obtiene la **regla del producto**: $P(A \cap B) = P(A | B) \cdot P(B)$. Aplicada iterativamente sobre una secuencia de n símbolos $x_1, x_2, \dots, x_n$, se deriva la **regla de la cadena exacta**:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \cdot P(x_2 | x_1) \cdot P(x_3 | x_1, x_2) \cdots P(x_n | x_1, \dots, x_{n-1})$$

La regla del producto, derivada directamente de la definición de probabilidad condicional, dice $P(A \cap B) = P(A | B) \cdot P(B)$. Aplicada en cadena a n sucesos da la regla de la cadena: $P(x_1, \dots, x_n) = P(x_1) \cdot P(x_2 | x_1) \cdots P(x_n | x_1, \dots, x_{n-1})$, el producto de cada símbolo condicionado a todos los anteriores.

1.3 Clasificar con Bayes

El objetivo de la clasificación en bioinformática es asignar una secuencia observada X a una de varias clases candidatas $C \in \{\text{Cod}, \text{NCod}\}$ (región codificante vs no codificante).

1.3.1 El Teorema de Bayes

El **teorema de Bayes** invierte la dirección de la inferencia:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)} = \frac{\text{Verosimilitud} \times \text{Probabilidad a Priori}}{\text{Evidencia Marginal}}$$

Donde:

- $P(C)$: Probabilidad a priori de la clase (proporción global en el genoma).
- $P(X | C)$: Verosimilitud de observar la secuencia X dentro de la clase C .
- $P(X) = \sum_{C'} P(X | C')P(C')$: Evidencia normalizadora, idéntica para todas las clases que compiten.

El teorema de Bayes calcula la probabilidad a posteriori de una clase a partir de tres cantidades estimables: la verosimilitud $P(\text{datos} | \text{clase})$, la probabilidad a priori $P(\text{clase})$ y la evidencia $P(\text{datos})$, que actúa como normalizador y es idéntica para todas las clases que compiten.

1.3.2 El supuesto «ingenuo» (Naïve Bayes)

Calcular la verosimilitud exacta $P(x_1, \dots, x_n | C)$ requeriría estimar las probabilidades de las 4^n posibles secuencias completas, lo cual es inviable. El clasificador ****Naïve-Bayes**** asume la hipótesis simplificadora de que los nucleótidos de la secuencia son condicionalmente independientes entre sí dada la clase:

$$P(x_1, x_2, \dots, x_n | C) \approx \prod_{i=1}^n P(x_i | C)$$

Aunque ignora el orden de las bases, Naïve-Bayes es extremadamente eficaz en la práctica porque solo necesita ordenar correctamente las clases candidatas, no estimar probabilidades exactas.

El supuesto ingenuo de Naïve Bayes asume que los atributos son condicionalmente independientes dada la clase, lo que descompone la verosimilitud en el producto de las probabilidades de cada atributo; esta independencia casi nunca es exacta —ignora el orden de las bases—, pero el método funciona bien porque solo necesita ordenar correctamente las clases, no estimar probabilidades exactas.

1.3.3 Regla de decisión y ejemplos resueltos

Dado que el denominador $P(X)$ es constante para todas las clases, la regla de decisión consiste en seleccionar la clase con mayor producto de a priori por verosimilitud:

$$\hat{C} = \arg \max_C \left[P(C) \prod_{i=1}^n P(x_i | C) \right]$$

La regla de decisión de Naïve Bayes calcula para cada clase candidata el producto de la probabilidad a priori por las verosimilitudes de cada atributo —descartando el denominador del teorema de Bayes por ser idéntico para todas las clases— y elige la clase que obtiene el valor más alto.

Consideremos un modelo entrenado con los siguientes parámetros:

- ****Clase Codificante (Cod):**** $P(\text{Cod}) = 0.45$; $P(A | \text{Cod}) = 0.4$, $P(C | \text{Cod}) = 0.2$, $P(G | \text{Cod}) = 0.3$, $P(T | \text{Cod}) = 0.1$.
- ****Clase No Codificante (NCod):**** $P(\text{NCod}) = 0.55$; $P(A | \text{NCod}) = 0.3$, $P(C | \text{NCod}) = 0.3$, $P(G | \text{NCod}) = 0.2$, $P(T | \text{NCod}) = 0.2$.

Caso 1: Clasificación de ‘AAGT’.

- $P(\text{Cod} | \text{AAGT}) \propto 0.45 \times 0.4 \times 0.4 \times 0.3 \times 0.1 = \mathbf{0.00216}$.
- $P(\text{NCod} | \text{AAGT}) \propto 0.55 \times 0.3 \times 0.3 \times 0.2 \times 0.2 = \mathbf{0.00198}$.

Gana la clase **Cod**, pero con un margen estrecho (0.00216 vs 0.00198), lo que refleja incertidumbre diagnóstica.

Con el modelo entrenado del ejemplo ($P(A|\text{Cod})=0.4$, $P(C|\text{Cod})=0.2$, $P(G|\text{Cod})=0.3$, $P(T|\text{Cod})=0.1$, $P(\text{Cod})=0.45$; $P(A|\text{NCod})=0.3$, $P(C|\text{NCod})=0.3$, $P(G|\text{NCod})=0.2$, $P(T|\text{NCod})=0.2$, $P(\text{NCod})=0.55$), clasificar AAGT da $P(\text{Cod}|\text{AAGT})$ proporcional a 0.00216 y $P(\text{NCod}|\text{AAGT})$ proporcional a 0.00198: gana Cod pero la decisión es ajustada, no una certeza. Se reproduce con `verification/chapter_results.sh naive_bayes_aagt`.

Caso 2: Clasificación de ‘CGT’.

- $P(\text{Cod} \mid \text{CGT}) \propto 0.45 \times 0.2 \times 0.3 \times 0.1 = \mathbf{0.0027}$.
- $P(\text{NCod} \mid \text{CGT}) \propto 0.55 \times 0.3 \times 0.2 \times 0.2 = \mathbf{0.0066}$.

Gana **NCod** con una holgura contundente ($\approx 2.44\times$ superior a Cod). Comparar el margen entre clases distingue decisiones sólidas de empates técnicos.

Sobre el mismo modelo entrenado, clasificar CGT da $P(\text{Cod}|\text{CGT})$ proporcional a 0.0027 y $P(\text{NCod}|\text{CGT})$ proporcional a 0.0066: gana NCod con un margen relativo mucho más amplio (NCod casi 2.4 veces Cod) que en el ejemplo AAGT —una decisión holgada, no un empate técnico—. Comparar el margen entre clases, y no solo la clase ganadora, distingue una decisión sólida de una dudosa. Se reproduce con `verification/chapter_results.sh naive_bayes_cgt`.

1.3.4 El problema de la frecuencia cero y el subdesbordamiento numérico

1. Suavizado de Laplace (+1). Si un nucleótido nunca apareció en los datos de entrenamiento de una clase, su frecuencia estimada es 0. Dado que la verosimilitud es un producto, ese único cero anula completamente el valor de la clase ($0 \times \dots = 0$), aunque todos los demás nucleótidos encajen a la perfección. Para solucionarlo, el **suavizado de Laplace** suma 1 a todos los conteos antes de normalizar:

$$P(x_i \mid C) = \frac{\text{Conteo}(x_i) + 1}{N_C + |\Sigma|}$$

donde $|\Sigma| = 4$ para el alfabeto de nucleótidos.

Si un nucleótido nunca apareció en el conjunto de entrenamiento de una clase, su probabilidad estimada es 0; como la verosimilitud es un producto, ese único factor cero anula por completo la clase, aunque el resto de atributos encajen perfectamente. El suavizado de Laplace lo corrige sumando 1 a cada conteo antes de calcular frecuencias, garantizando que ninguna probabilidad estimada sea exactamente 0. Se reproduce con `verification/chapter_results.sh laplace_smoothing_demo`.

2. Cómputo en espacio logarítmico. Al multiplicar cientos de probabilidades pequeñas ($P < 1$), el resultado cae por debajo de la precisión mínima de la coma flotante ($< 10^{-308}$), produciendo un ***subdesbordamiento numérico (*underflow*)** a cero. La solución estándar consiste en operar en el dominio logarítmico, transformando los productos en sumas:

$$\log P(C \mid X) \propto \log P(C) + \sum_{i=1}^n \log P(x_i \mid C)$$

Dado que el logaritmo es una función estrictamente monótona creciente, la clase que maximiza la suma de logaritmos es idéntica a la que maximiza el producto.

Multiplicar muchas probabilidades pequeñas —una verosimilitud por símbolo de una secuencia larga— provoca un subdesbordamiento a cero en coma flotante, perdiendo la información necesaria para decidir. Sumar logaritmos en lugar de multiplicar probabilidades evita el subdesbordamiento —el logaritmo convierte productos en sumas— y, al ser una función creciente, produce exactamente la misma clase ganadora que el cálculo sin logaritmos. Se reproduce con `verification/chapter_results.sh log_space_classification`.

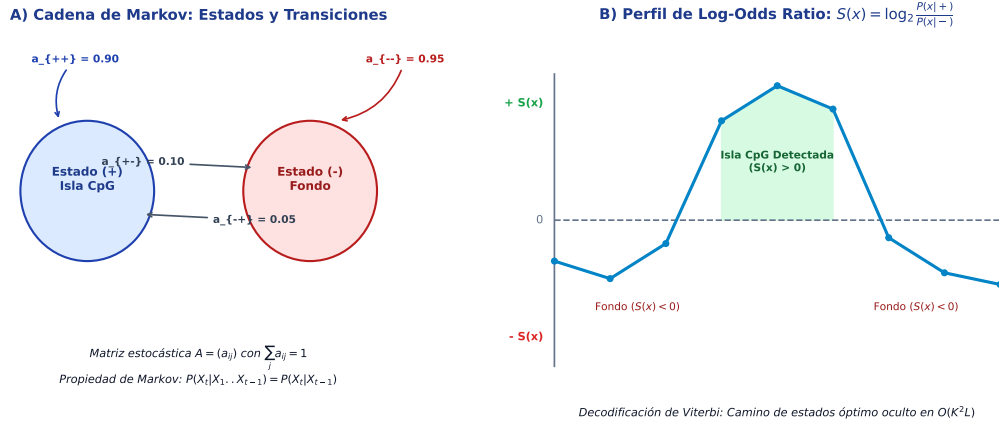


Figure 1.1: A) Modelo de Cadena de Markov discreta con estados y probabilidades de transición a_{ij} . B) Perfil de *log-odds ratio* $S(x) = \log_2 \frac{P(x|+)}{P(x|-)}$ para la discriminación estadística de islas CpG frente al fondo genómico neutro. Figura original adaptada de las presentaciones docentes.

1.4 Cadenas de Markov: recuperar el orden

1.4.1 La propiedad de Markov de orden 1

Para superar la ceguera de Naïve Bayes ante el orden de los nucleótidos, las **cadenas de Markov de orden 1** establecen que la probabilidad de un símbolo en la posición i depende exclusivamente del símbolo inmediatamente anterior x_{i-1} :

$$P(x_i | x_1, x_2, \dots, x_{i-1}) = P(x_i | x_{i-1})$$

La propiedad de Markov de orden 1 establece que la probabilidad del símbolo en la posición i depende únicamente del símbolo en la posición $i - 1$, no de toda la secuencia precedente; se sitúa a medio camino entre Naïve Bayes (orden 0, ninguna dependencia) y un modelo que condicionara a todas las posiciones anteriores.

1.4.2 Definición formal y validación de la matriz de transición

Una cadena de Markov discreta se define formalmente mediante tres elementos:

1. El conjunto de estados posibles $S = \{A, C, G, T\}$.
2. El vector de probabilidades iniciales $\pi = [P(A), P(C), P(G), P(T)]$, con $\sum \pi_i = 1$.
3. La **matriz de transición** A , donde cada elemento $a_{jk} = P(s_k | s_j)$ representa la probabilidad de transitar del estado s_j al s_k .

Invariante fundamental: Cada fila de la matriz de transición debe sumar estrictamente 1:

$$\sum_k P(s_k | s_j) = 1, \quad \forall j$$

Una cadena de Markov se define por tres componentes: los estados posibles, la probabilidad inicial de cada estado y una matriz de transición con las probabilidades condicionales de pasar de cada estado al siguiente. Cada fila de la matriz de transición debe sumar exactamente 1, porque desde un estado dado la cadena pasa con certeza a alguno de los estados posibles.

La probabilidad conjunta de una secuencia completa bajo una cadena de Markov es:

$$P(x_1, x_2, \dots, x_n) = P(x_1) \prod_{i=2}^n P(x_i | x_{i-1})$$

La probabilidad de una secuencia completa bajo una cadena de Markov de orden 1 es la probabilidad inicial del primer símbolo multiplicada por el producto de cada transición sucesiva. A diferencia de Naïve Bayes, donde cada factor es un $P(x_i)$ independiente, aquí cada factor es condicional al símbolo anterior; esa es la diferencia formal entre ignorar el orden y modelarlo.

Listing 1.1: Algoritmo formal en pseudocódigo para la evaluación de la verosimilitud de una secuencia en una Cadena de Markov en espacio logarítmico.

```

ALGORITMO: Cadena_Markov_Evaluacion_Probabilidad
ENTRADA:
- X: Secuencia observada de nucleotidos x_0, x_1, ..., x_{T-1} de longitud T
- pi: Vector de probabilidades iniciales pi[c] para cada c en Sigma
- A: Matriz estocastica de transicion A[origen, destino]
SALIDA:
- Log_Verosimilitud: log2(P(X | Modelo))
COMPLEJIDAD: Tiempo O(T), Espacio auxiliar O(1)

1. INICIALIZACION EN ESPACIO LOGARITMICO:
  Si pi[X[0]] <= 0 entonces:
    Retornar -INFINITO # Emision inicial imposible bajo el modelo

  Log_P <- log2(pi[X[0]])

2. PRODUCTO DE TRANSICIONES SUCESIVAS:
  Para t desde 1 hasta T - 1:
    previo <- X[t - 1]
    actual <- X[t]

    prob_transicion <- A[previo, actual]
    Si prob_transicion <= 0 entonces:
      Retornar -INFINITO # Transicion prohibida

    Log_P <- Log_P + log2(prob_transicion) # Suma de logaritmos previene underflow

3. RETORNO:
  Retornar Log_P

```

1.4.3 Trazas de cálculo y matrices inválidas

Consideremos el modelo de transición entre nucleótidos con $\pi(A) = 0.3$, $\pi(C) = 0.2$, $\pi(G) = 0.3$, $\pi(T) = 0.2$ y la matriz A :

$s_{i-1} \setminus s_i$	A	C	G	T
A	0.1	0.4	0.4	0.1
C	0.2	0.3	0.1	0.4
G	0.3	0.2	0.3	0.2
T	0.2	0.2	0.3	0.3

Traza 1: Probabilidad de ‘ACG’.

$$P(ACG) = P(A) \cdot P(C | A) \cdot P(G | C) = 0.3 \times 0.4 \times 0.1 = \mathbf{0.012}$$

Con probabilidad inicial $P(A) = 0.3$ y la matriz de transición del ejemplo resuelto, la probabilidad de ACG es $P(A) \cdot P(C | A) \cdot P(G | C) = 0.3 \times 0.4 \times 0.1 = 0.012$. Se reproduce con `verification/chapter_results.sh markov_trace_acg`.

Traza 2: Probabilidad de ‘CGT’.

$$P(\text{CGT}) = P(C) \cdot P(G | C) \cdot P(T | G) = 0.2 \times 0.1 \times 0.2 = \mathbf{0.004}$$

Con probabilidad inicial $P(C) = 0.2$ y la misma matriz de transición, la probabilidad de CGT es $P(C) \cdot P(G | C) \cdot P(T | G) = 0.2 \times 0.1 \times 0.2 = 0.004$. Se reproduce con `verification/chapter_results.sh markov_trace_cgt`.

Detección de matrices corruptas. Una fila como $[0.1, 0.5, 0.4, 0.2]$ suma $1.2 \neq 1.0$. No constituye una distribución probabilística válida y cualquier cálculo derivado de ella carece de validez matemática.

Una fila de matriz de transición $[0.1, 0.5, 0.4, 0.2]$ suma 1.2, no 1: no es una distribución de probabilidad válida y cualquier cálculo de probabilidad de secuencia que la use queda invalidado, por correcto que parezca el resultado. Validar que cada fila sume 1 debe hacerse antes de calcular, no tras obtener un resultado sospechoso. Se reproduce con `verification/chapter_results.sh validate_matrix_rows`.

1.4.4 Explosión de parámetros y detección de islas CpG

Explosión de parámetros en orden superior. Un modelo de Markov de orden k sobre un alfabeto de tamaño $|\Sigma|$ define $|\Sigma|^k$ contextos posibles y almacena $|\Sigma|^{k+1}$ probabilidades condicionales en su matriz. Puesto que las probabilidades de cada fila deben sumar obligatoriamente 1, cada contexto impone una ligadura matemática, dejando exactamente $|\Sigma|^k(|\Sigma| - 1)$ grados de libertad o parámetros libres independientes. Para ADN ($|\Sigma| = 4$) y $k = 3$, se tienen $4^3 = 64$ contextos, 256 probabilidades almacenadas y 192 parámetros libres independientes (más la distribución inicial). Con conjuntos de datos pequeños, la inmensa mayoría de los contextos no aparecen nunca, provocando sobreajuste o estimaciones nulas.

Un modelo de Markov de orden k sobre un alfabeto de 4 símbolos almacena 4^{k+1} probabilidades condicionales organizadas en 4^k contextos de 4 celdas cada uno, lo que supone $4^k \times 3$ grados de libertad independientes. Para $k = 3$ son 64 contextos, 256 probabilidades almacenadas y 192 parámetros libres; con conjuntos de entrenamiento reducidos, la mayoría de esos contextos apenas aparecerán, provocando sobreajuste o estimaciones nulas. Se reproduce con `verification/chapter_results.sh markov_parameter_scaling 3`.

Detección de islas CpG. Dos regiones con idéntica proporción de A, C, G y T resultan indistinguibles para Naïve Bayes (orden 0). Sin embargo, una cadena de Markov de orden 1 las discrimina directamente a través de la probabilidad $P(G | C)$, que es mucho más elevada en promotores e islas CpG desmetiladas que en el genoma general (donde la metilación de citosinas y posterior desaminación a timina provoca una depleción característica de dinucleótidos CG).

Dos regiones de ADN con idénticas frecuencias de A, C, G y T pueden diferir radicalmente en su proporción de transiciones C-seguida-de-G (islas CpG frente a ADN genómico general): un modelo de orden 0 (frecuencias sueltas como en Naïve Bayes) es ciego a esa diferencia porque nunca condiciona al símbolo anterior; un modelo de orden 1 la distingue directamente en la celda $P(G | C)$ de su matriz de transición, mucho más alta en una isla CpG que en el genoma general. Se reproduce con `verification/chapter_results.sh cp_g_markov_discrimination`.

1.5 Del estado observado al oculto (HMM)

En muchas aplicaciones genómicas reales, la propiedad funcional de una base (si pertenece a un **Promotor** o a una región **No Promotora**) no está escrita explícitamente: es un **estado oculto**.

Un **Modelo Oculto de Markov (HMM)** desacopla el sistema en dos capas:

1. **Capa oculta:** Una secuencia de estados no observables $q_1, q_2, \dots, q_n$ que evoluciona como una cadena de Markov con probabilidades iniciales π y matriz de transición entre estados $A = (a_{ij})$.
2. **Capa observada:** La secuencia de nucleótidos visibles $x_1, x_2, \dots, x_n$, emitida por cada estado según una matriz de emisión $B = (b_j(x_k))$.

Un modelo oculto de Markov (HMM) separa dos capas: los estados, que no se observan directamente y evolucionan como una cadena de Markov (con probabilidad inicial y matriz de transición entre estados), y las observaciones, los símbolos visibles que cada estado emite según su propia distribución de emisión. Un estado como «promotor» es una hipótesis del modelo, no una etiqueta visible en la molécula.

1.5.1 Los tres problemas clásicos de los HMM

La teoría de HMM plantea tres problemas fundamentales:

1. **Evaluación:** Calcular la probabilidad total de la secuencia observada $P(X | \lambda)$ mediante el **Algoritmo Forward** ($O(N^2T)$).
2. **Decodificación:** Determinar la secuencia óptima de estados ocultos $\hat{Q}$ que generó las observaciones mediante el **Algoritmo de Viterbi**.
3. **Aprendizaje:** Ajustar los parámetros del modelo a partir de secuencias no anotadas mediante el **Algoritmo de Baum-Welch (EM)**.

Sobre un HMM se plantean tres preguntas distintas: evaluación (probabilidad total de una secuencia de observaciones bajo el modelo, resuelta con el algoritmo forward), decodificación (secuencia de estados ocultos más probable, resuelta con el algoritmo de Viterbi) y aprendizaje (estimar parámetros a partir de datos, p. ej. Baum-Welch). Este capítulo desarrolla la decodificación con parámetros dados; el aprendizaje no supervisado queda fuera de alcance.

1.5.2 Definición del modelo HMM para promotores

Definimos un HMM de dos estados para la identificación de promotores génicos: $S = \{P, NP\}$ (Promotor y No Promotor), con $\pi(P) = 0.5$ y $\pi(NP) = 0.5$.

- **Matriz de transición entre estados (A):** Favorece la persistencia dentro del mismo estado: $P(P | P) = 0.7$, $P(NP | P) = 0.3$; $P(NP | NP) = 0.8$, $P(P | NP) = 0.2$.
- **Matriz de emisión de símbolos (B):** El estado promotor emite preferentemente motivos ricos en A y T (caja TATA): $b_P(A) = 0.4$, $b_P(T) = 0.3$, $b_P(C) = 0.1$, $b_P(G) = 0.2$. El estado no promotor emite de forma uniforme: $b_{NP}(A) = b_{NP}(C) = b_{NP}(G) = b_{NP}(T) = 0.25$.

En el HMM de dos estados para identificar promotores (P=promotor, NP=no promotor), la matriz de transición favorece la persistencia en ambos estados ($P(P|P)=0.7$, $P(NP|NP)=0.8$) frente al cambio ($P(NP|P)=0.3$, $P(P|NP)=0.2$); la matriz de emisión sesga el estado P hacia A y T ($P(A|P)=0.4$, $P(T|P)=0.3$ frente a $P(C|P)=0.1$, $P(G|P)=0.2$), mientras que NP emite las cuatro bases de forma equiprobable (0.25 cada una).

1.6 Trazar Viterbi

1.6.1 El Algoritmo de Viterbi (Programación Dinámica)

El algoritmo de Viterbi encuentra la secuencia de estados ocultos globalmente óptima que maximiza $P(Q, X | \lambda)$ mediante programación dinámica sobre la variable $\delta_t(j)$, que representa la probabilidad del camino más probable que termina en el estado j en el instante t :

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(x_t)$$

Simultáneamente, se registra en una matriz de punteros $\psi_t(j)$ el estado predecesor i que maximizó dicha probabilidad:

$$\psi_t(j) = \arg \max_{1 \leq i \leq N} [\delta_{t-1}(i) \cdot a_{ij}]$$

El algoritmo de Viterbi es programación dinámica sobre $\delta_t(j) = b_j(x_t) \cdot \max_i [\delta_{t-1}(i) \cdot a_{ij}]$: cada celda guarda la mejor puntuación de un camino de estados que termina en el estado j en el instante t , junto con un puntero al estado predecesor que logró esa puntuación. El camino completo se reconstruye retrocediendo los punteros desde el mejor estado final. En la práctica se calcula en logaritmos por la misma razón que Naïve Bayes: evitar el subdesbordamiento numérico.

Convención de indexación y pseudocódigo del Algoritmo de Viterbi: En este texto adoptamos la indexación estándar en computación $t = 0, \dots, T-1$ para una secuencia observada de longitud T ($x_0, x_1, \dots, x_{T-1}$):

1. **Inicialización** ($t = 0$): Para cada estado j , calcular $\delta_0(j) = \pi_j \cdot b_j(x_0)$ y registrar puntero inicial $\psi_0(j) = \perp$ (sin predecesor).
2. **Inducción recursiva** ($t = 1 \dots T-1$): Para cada instante t y cada estado destino j :

$$\delta_t(j) = \max_i [\delta_{t-1}(i) \cdot a_{ij}] \cdot b_j(x_t), \quad \psi_t(j) = \arg \max_i [\delta_{t-1}(i) \cdot a_{ij}]$$

3. **Terminación** ($t = T-1$): Identificar el mejor estado final $q_{T-1}^* = \arg \max_j \delta_{T-1}(j)$.
4. **Trazado hacia atrás (*Backtracking*)**: Recuperar recursivamente los estados previos $q_t^* = \psi_{t+1}(q_{t+1}^*)$ desde $t = T-2$ descendiendo hasta $t = 0$.

El pseudocódigo de Viterbi inicializa δ en $t = 0$ con la probabilidad inicial por la emisión del primer símbolo para cada estado, registra que allí no hay predecesor, avanza paso a paso eligiendo para cada estado el predecesor que maximiza la puntuación previa por la transición y multiplicando por la emisión actual, y finalmente reconstruye el camino completo retrocediendo por la cadena de punteros desde el estado final con mejor puntuación.

Listing 1.2: Algoritmo formal en pseudocódigo para la decodificación de Viterbi en HMMs en escala logarítmica.

```

ALGORITMO: HMM_Algoritmo_Viterbi_Decodificacion
ENTRADA:
- X: Secuencia observada x_0, x_1, ..., x_{T-1} de longitud T
- Estados: Conjunto de K estados ocultos {s_1, s_2, ..., s_K}
- pi: Probabilidades iniciales pi[k]
- A: Matriz de transicion A[i, j] = P(s_j | s_i)
- E: Matriz de emision E[j, c] = P(c | s_j)
SALIDA:

```

- Q_{optimo} : Secuencia de estados ocultos q_0^* , q_1^* , ..., q_{T-1}^* mas probable
 COMPLEJIDAD: Tiempo $O(K^2 * T)$, Espacio DP $O(K * T)$

1. INICIALIZACION ($t = 0$ en escala logaritmica):
 Para cada estado k en Estados:
 $\text{delta}[k, 0] \leftarrow \log_2(\pi[k]) + \log_2(E[k, X[0]])$
 $\text{psi}[k, 0] \leftarrow \text{NULO}$ # Sin puntero previo
2. INDUCCION RECURSIVA ($t = 1$ hasta $T - 1$):
 Para t desde 1 hasta $T - 1$:
 Para cada estado destino j en Estados:
 $\text{mejor_score} \leftarrow -\text{INFINITO}$
 $\text{mejor_pred} \leftarrow \text{NULO}$

 Para cada estado origen i en Estados:
 $\text{score_candidato} \leftarrow \text{delta}[i, t - 1] + \log_2(A[i, j])$
 Si $\text{score_candidato} > \text{mejor_score}$ entonces:
 $\text{mejor_score} \leftarrow \text{score_candidato}$
 $\text{mejor_pred} \leftarrow i$

 $\text{delta}[j, t] \leftarrow \text{mejor_score} + \log_2(E[j, X[t]])$
 $\text{psi}[j, t] \leftarrow \text{mejor_pred}$
3. TERMINACION Y RECUPERACION DEL MEJOR ESTADO FINAL:
 $q_{\text{optimo}}[T - 1] \leftarrow \text{argmax}_k(\text{delta}[k, T - 1])$
4. BACKTRACKING (Retroceso de punteros desde $T-2$ hasta 0):
 Para t desde $T - 2$ descendiendo hasta 0:
 $q_{\text{optimo}}[t] \leftarrow \text{psi}[q_{\text{optimo}}[t + 1], t + 1]$
5. RETORNO:
 Retornar q_{optimo}

1.6.2 Traza completa sobre la secuencia ‘ATGC’

Analicemos la decodificación de $X = \text{‘ATGC’}$ con el modelo de promotores:

- $t = 0$ ($x_0 = A$): $\delta_0(P) = 0.5 \times 0.4 = 0.20$; $\delta_0(NP) = 0.5 \times 0.25 = 0.125$.
- $t = 1$ ($x_1 = T$): $\delta_1(P) = \max(0.20 \times 0.7, 0.125 \times 0.2) \times 0.3 = 0.14 \times 0.3 = 0.042$;
 $\delta_1(NP) = \max(0.20 \times 0.3, 0.125 \times 0.8) \times 0.25 = 0.10 \times 0.25 = 0.025$.
- $t = 2$ ($x_2 = G$): $\delta_2(P) = \max(0.042 \times 0.7, 0.025 \times 0.2) \times 0.2 = 0.0294 \times 0.2 = 0.00588$;
 $\delta_2(NP) = \max(0.042 \times 0.3, 0.025 \times 0.8) \times 0.25 = 0.020 \times 0.25 = 0.0050$.
- $t = 3$ ($x_3 = C$): $\delta_3(P) = \max(0.00588 \times 0.7, 0.0050 \times 0.2) \times 0.1 = 0.004116 \times 0.1 =$
0.000412; $\delta_3(NP) = \max(0.00588 \times 0.3, 0.0050 \times 0.8) \times 0.25 = 0.0040 \times 0.25 =$
0.001000.

El estado final óptimo en $t = 3$ es **NP** ($\delta_3(NP) = 0.001000 > \delta_3(P) = 0.000412$). Al retroceder los punteros de la matriz ψ , se obtiene el camino global óptimo:

$$\hat{Q} = [NP, NP, NP, NP]$$

Decodificando ATGC con el HMM de promotores, la tabla de Viterbi da $\delta_3(P) = 0.000412$ y $\delta_3(NP) = 0.001$: el camino óptimo completo es NP NP NP NP, con todos los punteros apuntando al mismo estado en cada paso (no ocurre ninguna transición de estado a lo largo del camino óptimo). Se reproduce con `verification/chapter_results.sh viterbi_atgc_trace`.

1.6.3 El peligro del clasificador posicional ingenuo

Si un analista intentara clasificar la secuencia $X = \text{'ATGC'}$ nucleótido a nucleótido evaluando de forma aislada qué estado tiene mayor emisión en cada posición individual (ignorando las transiciones A):

- Posición 0 ('A'): $b_P(A) = 0.4 > b_{NP}(A) = 0.25 \implies \mathbf{P}$.
- Posición 1 ('T'): $b_P(T) = 0.3 > b_{NP}(T) = 0.25 \implies \mathbf{P}$.
- Posición 2 ('G'): $b_{NP}(G) = 0.25 > b_P(G) = 0.20 \implies \mathbf{NP}$.
- Posición 3 ('C'): $b_{NP}(C) = 0.25 > b_P(C) = 0.10 \implies \mathbf{NP}$.

El método ingenuo posicional predice erróneamente [P, P, NP, NP].

Este resultado demuestra la diferencia conceptual entre optimización local y global:

- El clasificador por emisiones responde a una regla local posicional y predice [P, P, NP, NP].
- El algoritmo de Viterbi maximiza la probabilidad conjunta de ****toda la trayectoria**** (decodificación MAP global) y produce [NP, NP, NP, NP], evitando penalizaciones de cambio de estado.

Delimitación formal: Viterbi no garantiza que cada estado de su ruta sea el estado marginal más probable en esa posición individual; esa segunda cuestión requiere probabilidades posteriores ($P(q_t = S_i | X)$) calculadas mediante el algoritmo **Forward-Backward** (posterior decoding), el cual queda fuera del alcance de este curso.

Clasificar ATGC posición por posición usando solo la emisión más probable en cada instante (ignorando transiciones) da P P NP NP —A y T favorecen individualmente a P—, lo que difiere del camino de Viterbi completo NP NP NP NP. El camino más probable de Viterbi no equivale a elegir en cada posición el estado con mayor emisión: las transiciones acoplan las decisiones y una opción localmente atractiva puede no pertenecer al camino globalmente óptimo. Se reproduce con `verification/chapter_results.sh viterbi_vs_naive_emission`.

1.7 La operación peligrosa: matrices no normalizadas y ceros

El fallo más destructivo en modelos probabilísticos ocurre cuando se introduce una matriz de transición donde una fila no suma exactamente 1 o cuando una probabilidad cero en una secuencia no suavizada cancela catastróficamente toda la cadena de decisión.

Protocolo preventivo en 3 pasos:

1. **Validación de invariantes:** Verificar con aserciones que cada fila de la matriz de transición y emisión sume 1.0 ± 10^{-6} .
2. **Aplicación universal de Laplace:** Aplicar siempre suavizado +1 en el entrenamiento con datos finitos.
3. **Espacio logarítmico estricto:** Ejecutar todas las recurrencias de Naïve Bayes, Markov y Viterbi con $\log(P)$ y sumas.

1.8 Actividad de depuración: errores en Viterbi

Analice el siguiente fragmento con tres errores conceptuales en la recurrencia de Viterbi:

Listing 1.3: Fragmento con tres errores en la implementación de Viterbi.

```
viterbi(observaciones, estados, pi, A, B):
  # Fallo 1: inicializacion suma probabilidades en vez de multiplicarlas
  para cada estado s:
    delta[0][s] <- pi[s] + B[s][observaciones[0]]

  para t de 1 hasta longitud(observaciones) - 1:
    para cada estado j:
      # Fallo 2: se toma la suma de transiciones en vez del maximo
      delta[t][j] <- suma_sobre_i(delta[t-1][i] * A[i][j]) * B[j][observaciones[t]]
      # Fallo 3: se olvida registrar el puntero predecesor psi[t][j]

  # Sin matriz psi no es posible realizar el backtracking
  devolver reconstruir_camino(delta)
```

1.9 Síntesis operativa y tablas de diagnóstico

1.9.1 Tabla de diagnóstico de síntomas en modelos probabilísticos

Síntoma observado	Causa probable	Comprobación y corrección
Probabilidad a posteriori devuelve 0 en todas las clases	Cero no suavizado en una frecuencia observada	Aplicar suavizado de Laplace (+1) a los conteos
La probabilidad de una secuencia larga se convierte en 0	Subdesbordamiento numérico (*underflow*)	Cambiar la implementación a espacio logarítmico
La suma de probabilidades de estados en un instante supera 1	Matriz de transición corrupta (fila $\neq$ 1.0)	Normalizar estrictamente cada fila antes de la inducción
Viterbi produce un camino distinto a la intuición posicional	Efecto de acoplamiento de las transiciones	Es el comportamiento correcto: Viterbi optimiza globalmente

1.9.2 Tabla de síntesis con preguntas de control

Requisito del pipeline	Modelo recomendado	Pregunta de control
Clasificación global de secuencias	Naïve Bayes	¿Se ha aplicado suavizado de Laplace y log-espacio?
Modelado de dinucleótidos (islas CpG)	Cadena de Markov (orden 1)	¿Suman 1 todas las filas de la matriz A ?
Segmentación y anotación de regiones funcionales	HMM (Viterbi)	¿Se realiza el backtracking sobre la matriz ψ ?

1.10 Glosario conceptual

- **Probabilidad condicional:** $P(A | B)$, probabilidad de A dado B .
- **Naïve Bayes:** clasificador basado en independencia condicional de atributos.
- **Suavizado de Laplace:** técnica de regularización (+1) contra frecuencias nulas.
- **Underflow:** pérdida de precisión por multiplicación de números cercanos a cero.
- **Cadena de Markov:** proceso estocástico con memoria de un paso ($P(x_i | x_{i-1})$).
- **Matriz de transición:** tabla estocástica donde cada fila suma 1.
- **HMM:** modelo de Markov con estados ocultos y emisiones observadas.
- **Algoritmo de Viterbi:** programación dinámica para decodificar el camino óptimo.
- **Backtracking:** recuperación hacia atrás de los estados óptimos mediante punteros ψ .
- **Caja TATA:** motivo consenso rico en A/T característico de promotores.

1.11 Conexión con el curso y siguientes pasos

El alineamiento por programación dinámica (TC-CH09) recupera la técnica de trazado hacia atrás (*backtracking*) de punteros vista en Viterbi, aplicada a comparar dos secuencias en lugar de decodificar estados ocultos de una sola; los HMMs en bioinformática de producción (predictores de genes, HMMER) son la extensión industrial de la maquinaria estudiada aquí; y las redes neuronales (TC-CH10) abordan el mismo problema de clasificación que Naïve Bayes aprendiendo representaciones más ricas. Estos tres desarrollos son propios de capítulos posteriores y no se desarrollan aquí.

1.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas docentes de la asignatura (20251120_bioinfo_4.pptx, Álvaro Serrano Navarro) y en los textos canónicos de Durbin et al. (*Biological Sequence Analysis*) y Cormen et al. Todos los resultados numéricos se verifican deterministamente mediante `verification/chapter_results.sh`.

Anexo práctico · Clasificar, modelar y decodificar con probabilidad

Pregunta, producto y separación de materiales

¿Puede clasificar una secuencia con Naïve-Bayes a mano y contrastarlo con un script, validar una matriz de transición antes de confiar en ella, decodificar la ruta de estados ocultos más probable de un HMM con Viterbi, y distinguir esa ruta global de lo que parecería mejor posición a posición? Este laboratorio responde que sí, sin necesitar ningún dato externo.

El producto individual contiene: la clasificación manual de Naïve-Bayes sobre ACT contrastada con el script, la corrección de una matriz de transición inválida, la ruta Viterbi completa de la secuencia ATGCGTATATCGC, el punto exacto de cambio de estado en la perturbación AAAACCCC, y los tres controles de caso límite, todo en `notas/respuestas.tsv`.

El anexo es autónomo e independiente: no depende de ningún fichero externo. Utiliza `practice_assets/create_workspace.sh` para crear el espacio de trabajo del alumno y `verification/chapter_results.sh` como herramienta de cómputo y auditoría.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Preparar espacio de trabajo	5	Directorio creado, herramienta comprobada
2 · Clasificar con Naïve-Bayes	15–20	Clasificación manual contrastada con script
3 · Validar cadena de Markov	15–20	Matriz corregida, probabilidad de secuencia
4 · Decodificar con Viterbi	20–25	Ruta completa de estados vs ingenua
5 · Perturbación (AAAACCCC)	10–15	Punto de cambio de estado localizado
6 · Controles de robustez	10–15	Tres casos límite verificados
7 · Verificación y empaquetado	5	Comprobación con <code>DELIVERY_OK</code>

La ruta única de este anexo (no hay separación esencial/estudio) suma 75–110 minutos y produce evidencia comprobable en cada bloque.

1 · Preparar el espacio de trabajo

Listing 1.4: Comprobación de entorno y espacio de trabajo.

```
bash --version
chmod +x practice_assets/create_workspace.sh
./practice_assets/create_workspace.sh TC08_entrega
cd TC08_entrega
```

2 · Clasificar con Naïve-Bayes

Antes de ejecutar nada, clasifique a mano la secuencia ACT con el modelo de la sección de clasificación con Bayes ($P(A \mid \text{Cod}) = 0.4$, $P(C \mid \text{Cod}) = 0.2$, $P(T \mid \text{Cod}) = 0.1$, $P(\text{Cod}) = 0.45$; $P(A \mid \text{NCod}) = 0.3$, $P(C \mid \text{NCod}) = 0.3$, $P(T \mid \text{NCod}) = 0.2$, $P(\text{NCod}) = 0.55$). Anote su predicción y compárela:

Listing 1.5: Contrastar la clasificación manual.

```
bash herramienta/chapter_results.sh naive_bayes_classify ACT
```

Clasificar ACT da $P(\text{Cod} \mid \text{ACT}) \propto 0.45 \cdot 0.4 \cdot 0.2 \cdot 0.1 = 0.0036$ y $P(\text{NCod} \mid \text{ACT}) \propto 0.55 \cdot 0.3 \cdot 0.3 \cdot 0.2 = 0.0099$: gana NCod, con una diferencia relativa amplia (NCod casi 2.75 veces Cod). Se reproduce con `verification/chapter_results.sh naive_bayes_classify ACT`.

3 · Validar y usar una cadena de Markov

La siguiente matriz de transición, propuesta por una IA, tiene un error:

Listing 1.6: Detectar la fila inválida.

```
bash herramienta/chapter_results.sh validate_transition_row
```

Anote qué fila falla, cuánto suma realmente, y por qué eso invalida cualquier cálculo posterior que la use. Después calcule la probabilidad de una secuencia nueva, GCAT, con la matriz válida de la sección de cadenas de Markov y probabilidad inicial $P(G) = 0.3$:

Listing 1.7: Probabilidad de una secuencia nueva.

```
bash herramienta/chapter_results.sh markov_sequence_prob GCAT
```

Con probabilidad inicial $P(G)=0.3$ y la matriz de transición de la sección de cadenas de Markov, la probabilidad de GCAT es $P(G) \cdot P(C \mid G) \cdot P(A \mid C) \cdot P(T \mid A) = 0.3 \cdot 0.2 \cdot 0.2 \cdot 0.1 = 0.0012$. Se reproduce con `verification/chapter_results.sh markov_sequence_prob GCAT`.

4 · Decodificar con Viterbi

Sobre el HMM de promotores presentado en teoría, decodifique la secuencia completa usada en la fuente original de este ejemplo, ATGCGTATATCGC —observe que contiene un fragmento TATA, que recuerda a una caja TATA real; antes de ejecutar, prediga si eso basta para que la ruta pase por el estado P—.

Listing 1.8: Decodificar la secuencia completa.

```
bash herramienta/chapter_results.sh viterbi_trace ATGCGTATATCGC
```

Decodificando ATGCGTATATCGC (13 símbolos) con el HMM de promotores, la ruta Viterbi completa es NP trece veces seguidas: ni siquiera el fragmento TATA que contiene basta para que la ruta óptima entre en el estado P, porque el coste acumulado de la transición más las emisiones de C/G intercaladas supera a lo que aportaría el sesgo de P hacia A/T. Reconocer un patrón "a ojo" (TATA) no sustituye ejecutar el modelo completo. Se reproduce con `verification/chapter_results.sh viterbi_trace ATGCGTATATCGC`.

5 · Perturbación: AAAACCCC

Decodifique ahora AAAACCCC —cuatro A seguidas de cuatro C— y localice en qué posición exacta cambia de estado la ruta óptima.

Listing 1.9: Localizar el punto de cambio de estado.

```
bash herramienta/chapter_results.sh viterbi_trace AAAACCCC
```

Decodificando AAAACCCC, la ruta Viterbi es P,P,P,P,NP,NP,NP,NP: cambia de estado exactamente en la posición 4, donde empiezan las C. A diferencia de ATGCGTATATCGC, esta entrada sí produce un cambio de estado real, porque las cuatro A consecutivas acumulan suficiente ventaja para P antes de que las C fueren el cambio a NP. Se reproduce con `verification/chapter_results.sh viterbi_trace AAAACCCC`.

Anote en `notas/respuestas.tsv` en qué posición ocurre el cambio y qué parámetro del modelo (transición o emisión) es responsable.

6 · Controles de robustez

Listing 1.10: Comprobar casos limite.

```
bash herramienta/chapter_results.sh naive_bayes_classify ""
bash herramienta/chapter_results.sh viterbi_trace A
bash herramienta/chapter_results.sh laplace_smoothing_demo
```

Clasificar una secuencia vacía con Naïve-Bayes no falla: el producto vacío deja solo la comparación de las prioris, y gana NCod ($0.55 > 0.45$) por defecto. Decodificar una observación de un único símbolo con Viterbi no requiere ninguna transición: la ruta es el estado con mayor δ_0 , el mismo cálculo que la clasificación ingenua posición a posición para ese único instante. El suavizado de Laplace, ejecutado sin argumentos, reproduce el caso ancla de la sección de clasificación con Bayes de forma determinista. Se reproduce con `verification/chapter_results.sh naive_bayes_classify, viterbi_trace y laplace_smoothing_demo`.

7 · Verificar la entrega

Listing 1.11: Comprobar que la entrega esta completa antes de empaquetar.

```
cd ..
../practice_assets/check_practice_delivery.sh TC08_entrega
tar -czf TC08_entrega.tar.gz TC08_entrega
sha256sum TC08_entrega.tar.gz
```

`check_practice_delivery.sh` verifica, con Bash y GNU Coreutils exclusivamente, que `notas/respuestas.tsv` existe con la cabecera esperada y al menos una fila rellena, y que `herramienta/chapter_results.sh` está presente y es ejecutable.

Rúbrica de calificación ponderada

La evaluación de este anexo pondera la verificación técnica, el modelado probabilístico y la defensa conceptual sobre 10 puntos (reparto 30/70 según ADR-001):

- **3,0 puntos · Entrega técnica y verificador (DELIVERY_OK):** Entrega íntegra del paquete comprimido `.tar.gz`, formato requerido de `notas/respuestas.tsv` y superación de `check_practice_delivery.sh`.

- **2,5 puntos · Clasificación Naïve Bayes e invariantes de Markov:** Cálculo manual de verosimilitudes con log-probabilidades, suavizado de Laplace y comprobación estricta de que las filas de la matriz sumen 1.0.
- **2,5 puntos · Algoritmo de Viterbi y sensibilidad de parámetros:** Decodificación completa de la ruta MAP, identificación del punto de transición de estado en AAAACCCC y control de robustez ante entradas límite.
- **2,0 puntos · Defensa conceptual y acoplamiento global:** Justificación rigurosa (100–150 palabras) sobre el papel de las transiciones en Viterbi y por qué una caja TATA aislada no basta para transicionar a estado promotor sin el soporte de persistencia del modelo.

Nota didáctica: Obtener DELIVERY_OK acredita la correcta ejecución del flujo de trabajo, pero no sustituye la fundamentación formal de la probabilidad ni la defensa analítica de la decodificación.

Lista de autocorrección

- Mi clasificación manual de ACT coincidió con el script, o anoté por qué no.
- Detecté qué fila de la matriz de transición no sumaba 1 y por qué invalida los cálculos.
- Calculé la probabilidad de GCAT con la matriz corregida.
- Registré la ruta Viterbi completa de ATGCGTATATCGC y su contraste con TATA.
- Localicé la posición exacta de cambio de estado en AAAACCCC.
- Los tres controles de caso límite están anotados sin fallos.
- Puedo explicar por qué la ruta Viterbi puede diferir de clasificar posición a posición.
- `check_practice_delivery.sh` termina con DELIVERY_OK.

Defensa conceptual

En 100–150 palabras, explique por qué la secuencia ATGCGTATATCGC —que contiene un fragmento TATA— se decodifica enteramente como no promotor bajo este modelo, y qué tendría que cambiar en los parámetros (transición, emisión, o ambos) para que la ruta óptima sí pasara por el estado P en algún tramo.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes de bioinformática (20251120_bioinfo_4.pptx) y se valida al 100% mediante `verification/chapter_results.sh`.

Bibliography

- [1] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. 3rd edition, 2018. Encuadre bioinformatico de la programacion dinamica; citado hacia TC-CH09.
- [2] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998. Referencia canonica para cadenas de Markov, HMM y el algoritmo de Viterbi aplicados a secuencias biologicas.
- [3] M. Gardiner-Garden and M. Frommer. CpG islands in vertebrate genomes. *Journal of Molecular Biology*, 196(2):261–282, 1987.